Group 9

Data Models and Module Diagrams

Alex Driscoll, Sai Atluri, Aidan Jiang, Krish Arekar, Tanishq Saria

Project Manager: Preston

Framework: PERN Model

Database Management System:
Supabase - PostgreSQL

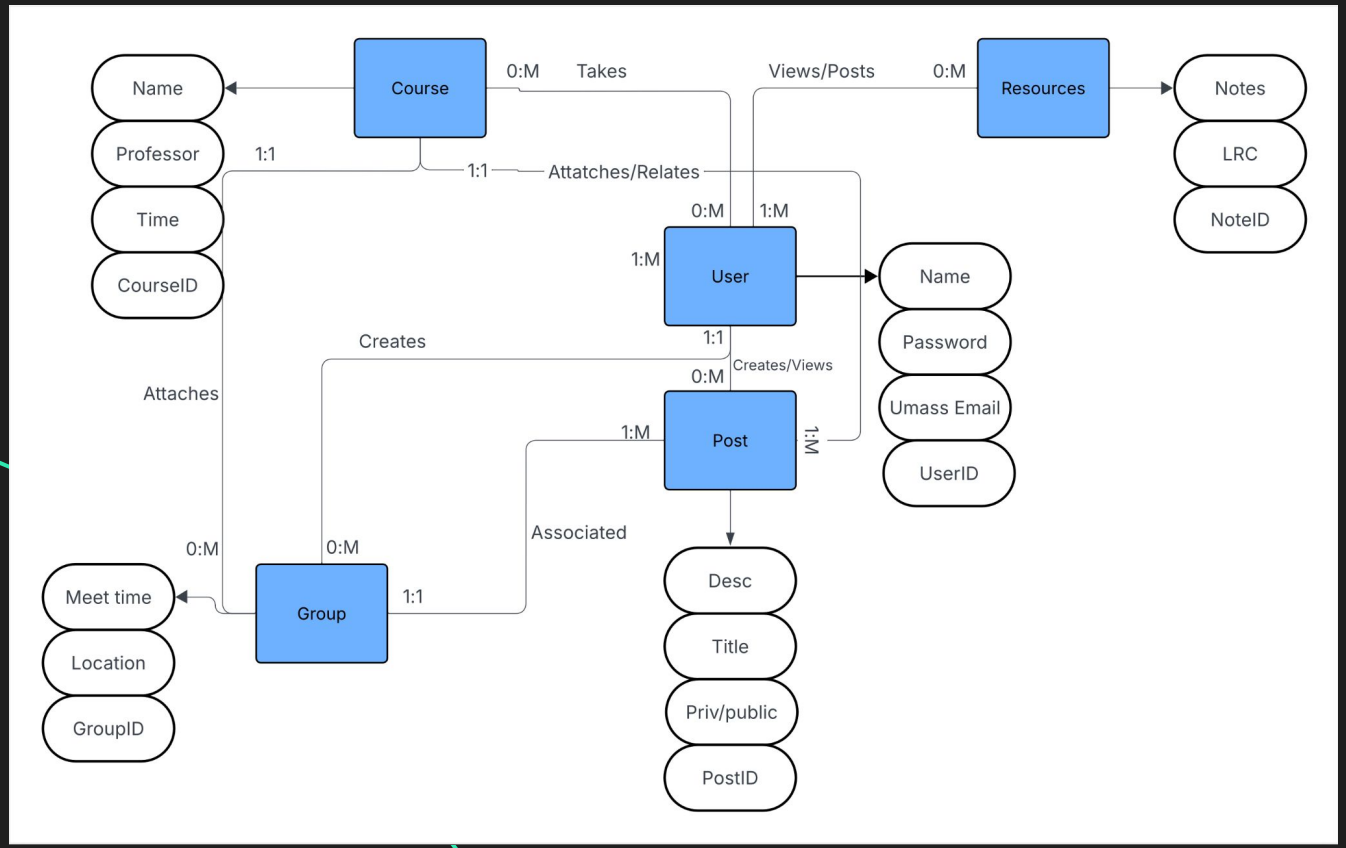
API:

Express

Frontend:
React

Backend:
Node.js

Data Model Diagram



Use Cases

01 Login Slide 05

02 Creating Posts Slide 08

03 Forming Groups Slide 12

04 Calendar Section Slide 16

05 Resource Section Slide 19

Login - Endpoint routes

localhost/api/auth/login

Authenticates users by calling auth.login()

localhost/api/auth/register

Creates new users by calling auth.register()

localhost/api/auth/logout

Ends session by calling auth.logout()

Login Section: Function Signatures

Function Signatures:

- auth.login(email: string, password: string): Promise<User>
- auth.register(name: string, email: string, password: string): Promise<User>
- auth.logout(userID: string): Promise<void>

Login Section: Testing Functions

auth.login():

- Populate users table with test user (email: test@umass.edu, password: hashed_pw)
- Call auth.login("test@umass.edu", "correct_password") → Returns user object
- Call auth. login("test@umass.edu", "wrong_password") → Returns error/null

auth.register():

- Call auth.register("John Doe" "john@umass. edu", "password123")
- Expected: New user created in database, returns user object with user ID
- Verify by checking users table in Supabase

Make a Post - Endpoint routes

localhost/getPosts

Will retrieve all posts by calling post.getAll(). What runs when the user first navigates to the posts page to see other people's study posts.

localhost/posts

Edits an existing post by calling post.updatePost(id: int, updates: PostData).

localhost/postCreatePosts

Calls post post.createPost(userID: int, subject: string, body: string, groupID: int)
Creates new post and saves to database

localhost/posts/{:id}

Deletes post if user by calling post.deletePost(id: int, userID: int) when id ={id}.

Posts Section: Function Signatures

Function Signatures:

- post.getAll() : Post[]
- post.createPost(userID: int, subject: string, body: string, groupID: int) : Post
- post.deletePost(id: int, userID: int) : boolean
- post.updatePost(id: int, updates: PostData) : Post

Post Section: Testing Functions

post.getAll()

- Populate post table with data

postid	subject	body	userid
1	Cs 320	“Final”	123
2	Math 233	“Lab”	124

- Call post.getAll()
- Function returns [{1, "CS 320", "Final", 1}, {2, "Math 235", "Lab", 2}]

post.createPost(userID: int, subject: string, body: string, groupID: int)

- User fills out post form on the dashboard and clicks Submit
- Clicking Submit accesses endpoint localhost/posts/create
- post.createPost() runs
- We can verify the post was stored by checking the posts table directly in the Supabase table editor

Post Section: Testing Functions

post.deletePost(id: int, userID: int)

- Call post.deletePost(1, 1) where postID == 1 and userID == 1 (the owner)
- Function returns true and post is removed
- Call post.deletePost(1, 2) where userID == 2 (not the owner)
- Function returns "unauthorized" and post remains in table

post.updatePost(id: int, updates: PostData)

- Call post.updatePost(1, {subject: "CS 326", body: "Updated study session"})
- Function returns the updated Post object with the new fields
- Verify by calling post.getAll() and confirming post 1 now shows the updated subject and body

Form a group - Endpoint routes

localhost/api/groups

Returns a JSON object with the newly created group's details, including its unique ID, name, and creator.

localhost/api/groups/{id}

Returns a JSON object containing the full details of the group with the specified ID, including an array of all its member objects.

localhost/api/groups/{id}/join

Returns a success confirmation JSON after adding the current user to the group with the specified ID.

localhost/api/groups/{id}/leave

Returns a success confirmation JSON after removing the current user from the group with the specified ID.

Group Formation: Function Signatures

Function Signatures:

- group.createGroup(ownerId: string, data: { name: string, course: string, maxSize?: number }): Promise<Group>
- group.joinGroup(userId: string, groupId: string): Promise<Membership>
- group.leaveGroup(userId: string, groupId: string): Promise<void>
- group.getGroupMembers(groupId: string): Promise<User[]>
- group.searchGroups(filters: { subject?: string, openOnly?: boolean }): Promise<Group[]>

Group Formation: Testing Functions

group.createGroup(ownerId, data: { name, course, maxSize})

- If required fields (name, course) are missing the group is not created
- If the ownerId is invalid the request fails
- If the input is valid a new group is created with the provided data
- If maxSize is not provided a default size is used

group.joinGroup(userId, groupId)

- If the groupId is invalid or the group does not exist it returns an error
- If the user is already a member it does not add them again
- If the group is full the user cannot join
- If valid the user is added and membership information is returned

group.leaveGroup(userId, groupId)

- If the groupId is invalid it returns an error
- If the user is not a member nothing changes
- If the user is a member they are removed from the group
- The function completes without returning data

Group Formation: Testing Functions

group.getGroupMembers (groupId)

- If the groupId is invalid it returns an empty list
- If the group exists it returns all current members
- If the group has no members it returns an empty list

group.searchGroups(filters: { subject, openOnly })

- If no filters are provided it returns all groups
- If subject is specified it returns groups for that subject
- If openOnly is true it returns only groups with available space
- If no groups match the filters it returns an empty list

Calendar Section: Endpoint Routes

localhost/getSIEvents/{:userid}

Retrieves an array of SI names, session locations and dates relating to a users courses by calling calendar.getSIEvents(userid: string)

localhost/getTutorEvents/{:userid}

Retrieves an array of tutor names, session locations and dates relating to a users courses by calling calendar.getTutorEvents(userid: string)

localhost/getGroupEvents/{:userid}

Retrieves the meeting times and locations of all groups that a user is added to by calling calendar.getGroupEvents(userid: string)

Calendar Section: Function Signatures

Function Signatures:

- calendar.getSIEvents(userid: string): CalendarEvent[]
- calendar.getTutorEvents(userid: string): CalendarEvent[]
- calendar.getGroupEvents(userid: string): CalendarEvent[]

interface CalendarEvent:

- title: string
- date: string
- timeStart: string
- timeEnd: string
- location: string

Calendar Section: Testing Functions

calendar.getSIEvents (userid: string)

- If the class is not supported return an empty list
- If the class is supported it returns the corresponding SI sessions
- If a class has multiple SI sessions available then all available SI sessions show up
- If the course input is not recognized it returns an empty list

calendar.getTutor Events(userid: string)

- If the class is not supported return an empty list
- If the class is supported it returns the corresponding tutor(s) information
- If the course input is not recognized it returns an empty list

calendar.getGroup Events(userid: string)

- If a user is in no groups then it returns an empty list
- If the id is negative or invalid it returns an empty list
- If the id is valid then it returns the corresponding group's information
- If a user is pending joining a group and it's id is input, then it does not reveal group information

Resources Section: Endpoint Routes

localhost/listAllNotes

Will retrieve all notes by calling notes.listAll()

localhost/listSpecificNotes/{:string}

Gets specific notes by calling notes.listSpecificNotes(s:string)

localhost/deleteNote/{:id}

Deletes a note by calling notes.deleteNote(id: string)

localhost/createNote{:note, :id}

Creates a note by calling notes.createNote(note: NoteObject, id: string)

Resource Section: Function Signatures

Function Signatures:

- notes.listAll()
- notes.listSpecificNotes(s:string)
- notes.deleteNote(id: string)
- notes.createNote(note: NoteObject, id: string)

Resource Section: Testing Functions

notes.listSpecificNotes(s:string)

- If there is no results, then empty array should be returned
- All tagged notes should appear

notes.deleteNote(id : string)

- Only user can delete own notes
- Other users should not be able to see the notes anymore

notes.createNote(note: NoteObject, id: string)

- Only user can create notes
- When note is created other users should be able to see notes

Thank you!